

Travaux à rendre : Analyse de Fourier & Équations Différentielles

Table of contents

Introduction	1
Partie 1 : Analyse de Fourier (TP1)	1
1.1 Définitions	1
1.2 Résultats graphiques	2
Partie 2 : Équations Différentielles (TP3)	3
2.1 Préparation	3
2.2 Pêche sans quota	4
2.3 Pêche avec quota	5

Introduction

Ce document regroupe les deux exercices à rendre demandés dans le fascicule “TP cpt maths”:

1. **TP1 (Analyse de Fourier)** : Étude de la convergence pour une fonction affine sur $[-\pi, \pi]$.
2. **TP3 (Équations Différentielles)** : Modélisation de la dynamique de population de poissons (pêche avec et sans quota).

Partie 1 : Analyse de Fourier (TP1)

Objectif : Tracer la série de Fourier de la fonction 2π -périodique définie sur $[-\pi, \pi]$ par $f(x) = \alpha x + 1$. Nous choisissons arbitrairement $\alpha = 1.572$ (valeur à 3 chiffres comprise entre 1 et 2).

1.1 Définitions

```
# 1. Paramètres
alpha <- 1.572

# 2. Fonction cible sur [-pi, pi]
```

```

f_target <- function(x) { alpha * x + 1 }

# 3. Calcul des coefficients de Fourier complexes sur [-pi, pi]
# Formule : cn = (1/2pi) * integrale(f(x) * exp(-inx))
coeff_pi <- function(f, n) {
  g <- function(x) { f(x) * exp(-1i * n * x) }

  # Intégration partie réelle et imaginaire séparément
  reg <- function(x) { Re(g(x)) }
  img <- function(x) { Im(g(x)) }

  int_re <- integrate(reg, -pi, pi)$value
  int_im <- integrate(img, -pi, pi)$value

  return( (1 / (2 * pi)) * (int_re + 1i * int_im) )
}

# 4. Somme partielle de Fourier d'ordre N
SN_pi <- function(f, N, x) {
  S <- coeff_pi(f, 0) # Terme constant

  for(n in 1:N) {
    cn <- coeff_pi(f, n)
    c_minus_n <- coeff_pi(f, -n)
    # Ajout des termes conjugués pour n et -n
    S <- S + cn * exp(1i * n * x) + c_minus_n * exp(-1i * n * x)
  }
  return(Re(S)) # On ne garde que la partie réelle
}

```

1.2 Résultats graphiques

Le graphique ci-dessous montre la convergence de la série vers la fonction cible pour $\alpha = 1$. Notez le phénomène de Gibbs aux bornes dû à la discontinuité de la fonction périodisée.

```

# Discrétisation
x_vals <- seq(-pi, pi, length.out = 500)
y_exact <- f_target(x_vals)

# Calcul des approximations (peut prendre quelques secondes)
y_N1 <- sapply(x_vals, function(x) SN_pi(f_target, 1, x))
y_N5 <- sapply(x_vals, function(x) SN_pi(f_target, 5, x))
y_N20 <- sapply(x_vals, function(x) SN_pi(f_target, 20, x))

# Tracé
plot(x_vals, y_exact, type = "l", lwd = 2, col = "black",
     xlab = "x", ylab = "y", ylim = range(c(y_exact, y_N20)),
     main = paste("Série de Fourier pour alpha =", alpha))

lines(x_vals, y_N1, col = "blue", lty = 2)
lines(x_vals, y_N5, col = "green", lty = 1)
lines(x_vals, y_N20, col = "red", lty = 1)

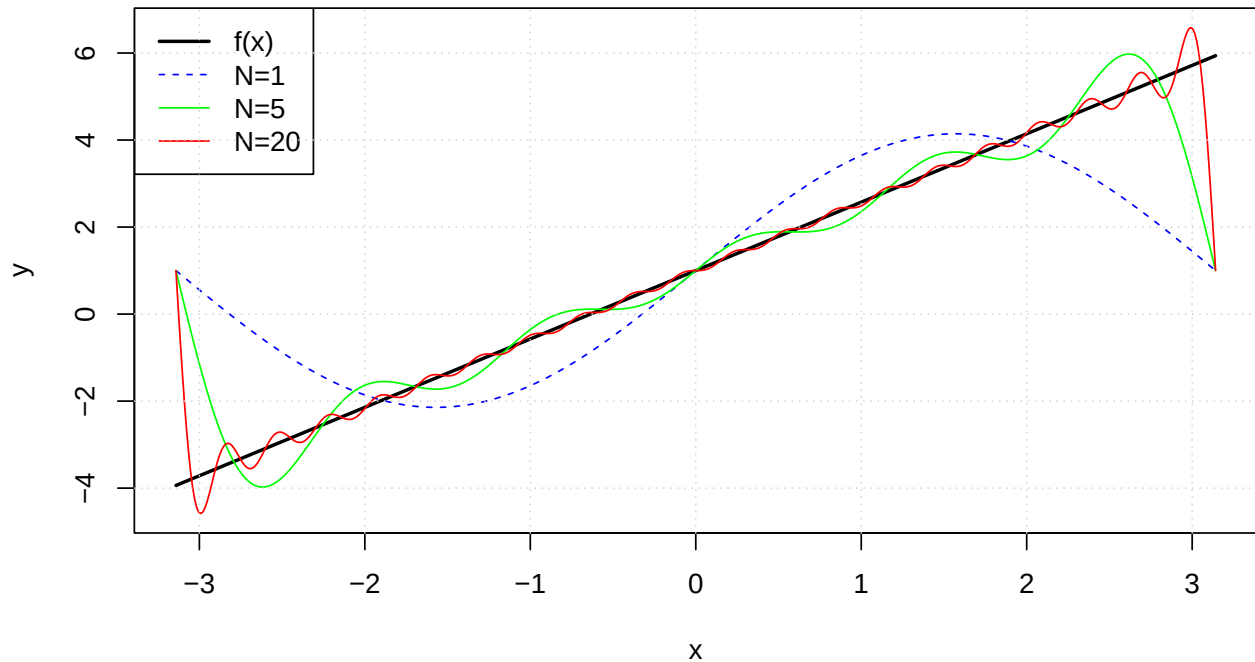
```

```

legend("topleft", legend = c("f(x)", "N=1", "N=5", "N=20"),
      col = c("black", "blue", "green", "red"),
      lty = c(1, 2, 1, 1), lwd = c(2, 1, 1, 1))
grid()

```

Série de Fourier pour $\alpha = 1.572$



Partie 2 : Équations Différentielles (TP3)

Nous utilisons ici le package `deSolve` pour résoudre numériquement les EDO.

2.1 Préparation

```

library(deSolve)

# Fonction utilitaire pour tracer le champ de directions
# (Permet de visualiser les tangentes des trajectoires)
champ <- function(xmax, pas, f, params) {
  plot(c(0, xmax*3), c(0, 1.5), type = "n", xlab = "Temps (t)", ylab = "Population (y)")
  t_val <- seq(0, xmax*3, pas)
  y_val <- seq(0, 1.5, pas/10) # Zoom sur les populations faibles

  points <- expand.grid(t_val, y_val)

  # Calcul du vecteur dérivé en chaque point

```

```

dy <- mapply(function(t, y) f(t, y, params), points[,1], points[,2])

# Normalisation pour l'affichage des flèches
scale <- pas / 8
arrows(points[,1], points[,2],
        points[,1] + scale, points[,2] + scale * dy,
        length = 0.02, col = "gray80")
}

```

2.2 Pêche sans quota

Modèle : . Paramètres : , , (choisi). Scénarios : Effort de pêche et .

```

# Définition du modèle (retourne une liste pour ode, scalaire pour champ)
f_sans_quota <- function(t, y, p) {
  val <- p["r"] * y * (1 - y/p["K"]) - p["E"] * y^2
  return(val)
}
Eq_sans_quota <- function(t, y, p) { list(f_sans_quota(t, y, p)) }

# Paramètres
y0 <- 1; K <- 5; r <- 0.08
discr <- seq(0, 30, 0.1)

# Résolution
pE1 <- c(r=r, K=K, E=0.2)
pE2 <- c(r=r, K=K, E=0.9)

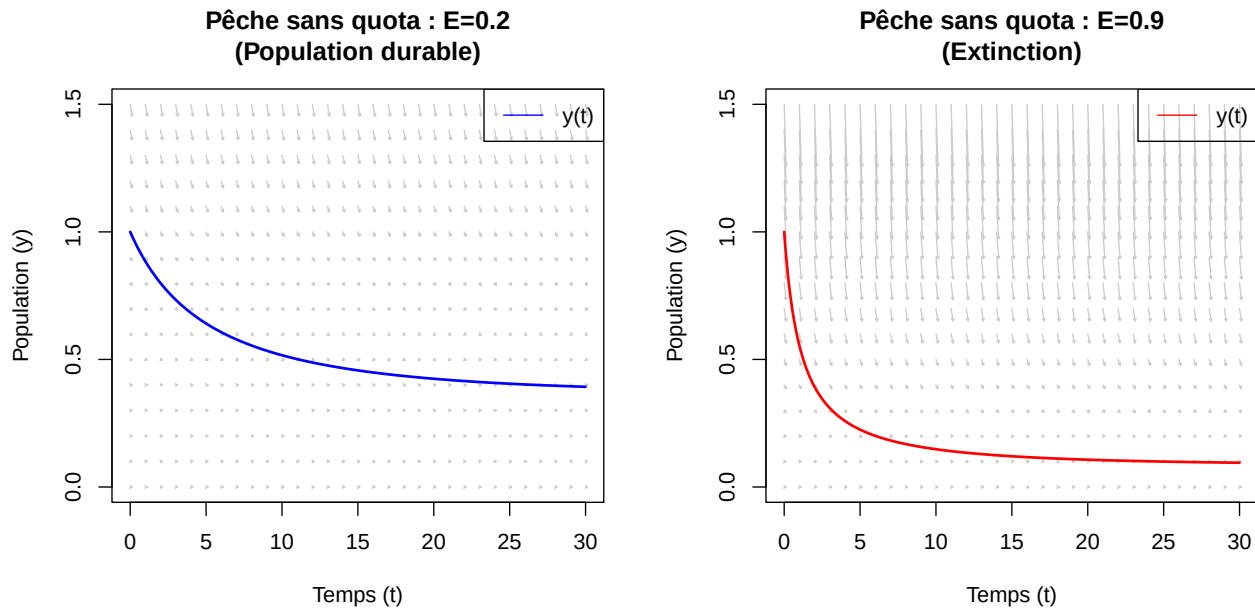
solE1 <- ode(y=y0, times=discr, func=Eq_sans_quota, parms=pE1)
solE2 <- ode(y=y0, times=discr, func=Eq_sans_quota, parms=pE2)

# Affichage côte à côte
par(mfrow = c(1, 2))

# Graphe E=0.2
champ(10, 1, function(t,y,p) f_sans_quota(t,y,p), pE1)
lines(solE1, col = "blue", lwd = 2)
title(main = "Pêche sans quota : E=0.2\n(Population durable)")
legend("topright", "y(t)", col="blue", lty=1)

# Graphe E=0.9
champ(10, 1, function(t,y,p) f_sans_quota(t,y,p), pE2)
lines(solE2, col = "red", lwd = 2)
title(main = "Pêche sans quota : E=0.9\n(Extinction)")
legend("topright", "y(t)", col="red", lty=1)

```



2.3 Pêche avec quota

Modèle : . Paramètres : (choisi entre 0.4 et 0.6), . Scénarios : Quota (élevé) et (faible).

```
# Définition du modèle
f_quota <- function(t, y, p) {
  val <- p["r"] * y * (1 - y/p["K"]) - p["Q"]
  return(val)
}
Eq_quota <- function(t, y, p) { list(f_quota(t, y, p)) }

# Paramètres
r_q <- 0.5
pQ1 <- c(r=r_q, K=K, Q=0.8)
pQ2 <- c(r=r_q, K=K, Q=0.1)

# Résolution
solQ1 <- ode(y=y0, times=discr, func=Eq_quota, parms=pQ1)
```

```
DLSODA- Warning..Internal T (=R1) and H (=R2) are
      such that in the machine, T + H = T on the next step
      (H = step size). Solver will continue anyway.
In above message, R1 = 5.46334, R2 = 3.75373e-16
```

```
DLSODA- Warning..Internal T (=R1) and H (=R2) are
      such that in the machine, T + H = T on the next step
      (H = step size). Solver will continue anyway.
In above message, R1 = 5.46334, R2 = 3.75373e-16
```

```
DLSODA- Warning..Internal T (=R1) and H (=R2) are
      such that in the machine, T + H = T on the next step
      (H = step size). Solver will continue anyway.
In above message, R1 = 5.46334, R2 = 3.10936e-16
```

DLSODA- Warning..Internal T (=R1) and H (=R2) are
such that in the machine, $T + H = T$ on the next step
(H = step size). Solver will continue anyway.
In above message, R1 = 5.46334, R2 = 3.10936e-16

DLSODA- Warning..Internal T (=R1) and H (=R2) are
such that in the machine, $T + H = T$ on the next step
(H = step size). Solver will continue anyway.
In above message, R1 = 5.46334, R2 = 3.10936e-16

DLSODA- Warning..Internal T (=R1) and H (=R2) are
such that in the machine, $T + H = T$ on the next step
(H = step size). Solver will continue anyway.
In above message, R1 = 5.46334, R2 = 2.48562e-16

DLSODA- Warning..Internal T (=R1) and H (=R2) are
such that in the machine, $T + H = T$ on the next step
(H = step size). Solver will continue anyway.
In above message, R1 = 5.46334, R2 = 2.48562e-16

DLSODA- Warning..Internal T (=R1) and H (=R2) are
such that in the machine, $T + H = T$ on the next step
(H = step size). Solver will continue anyway.
In above message, R1 = 5.46334, R2 = 2.05893e-16

DLSODA- Warning..Internal T (=R1) and H (=R2) are
such that in the machine, $T + H = T$ on the next step
(H = step size). Solver will continue anyway.
In above message, R1 = 5.46334, R2 = 2.05893e-16

DLSODA- Warning..Internal T (=R1) and H (=R2) are
such that in the machine, $T + H = T$ on the next step
(H = step size). Solver will continue anyway.
In above message, R1 = 5.46334, R2 = 2.05893e-16

DLSODA- Above warning has been issued I1 times.
It will not be issued again for this problem.
In above message, I1 = 10

DLSODA- At T (=R1), too much accuracy requested
for precision of machine.. See TOLSF (=R2)
In above message, R1 = 5.46334, R2 = nan

Warning in lsoda(y, times, func, parms, ...): Excessive precision requested.
scale up `rtol' and `atol' e.g by the factor 10

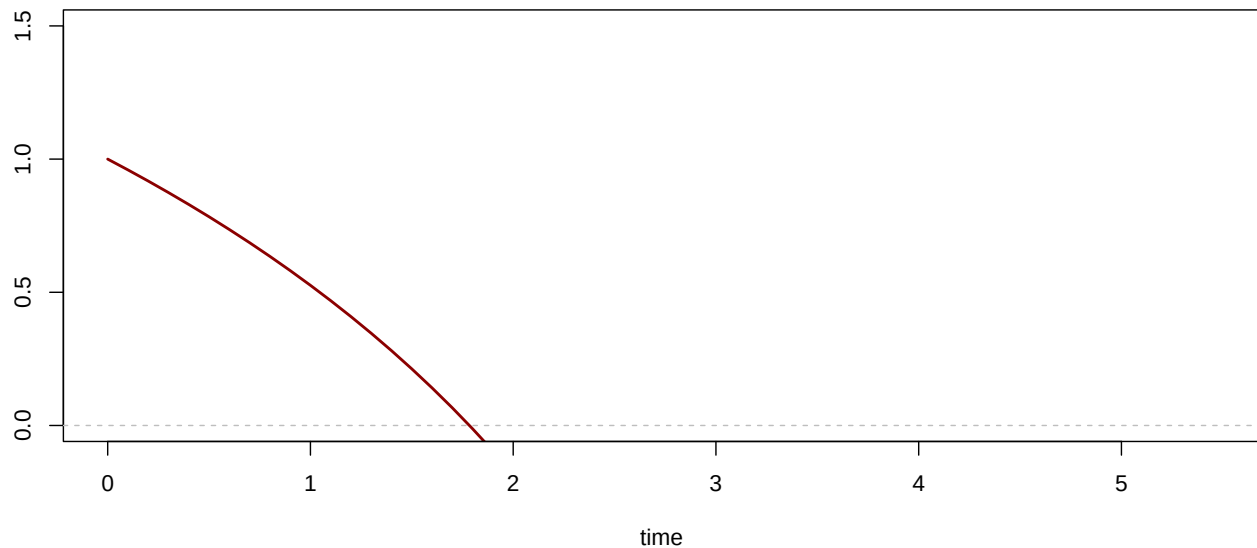
Warning in lsoda(y, times, func, parms, ...): Returning early. Results are
accurate, as far as they go

```
solQ2 <- ode(y=y0, times=discr, func=Eq_quota, parms=pQ2)
```

```
# Affichage
par(mfrow = c(1, 2))

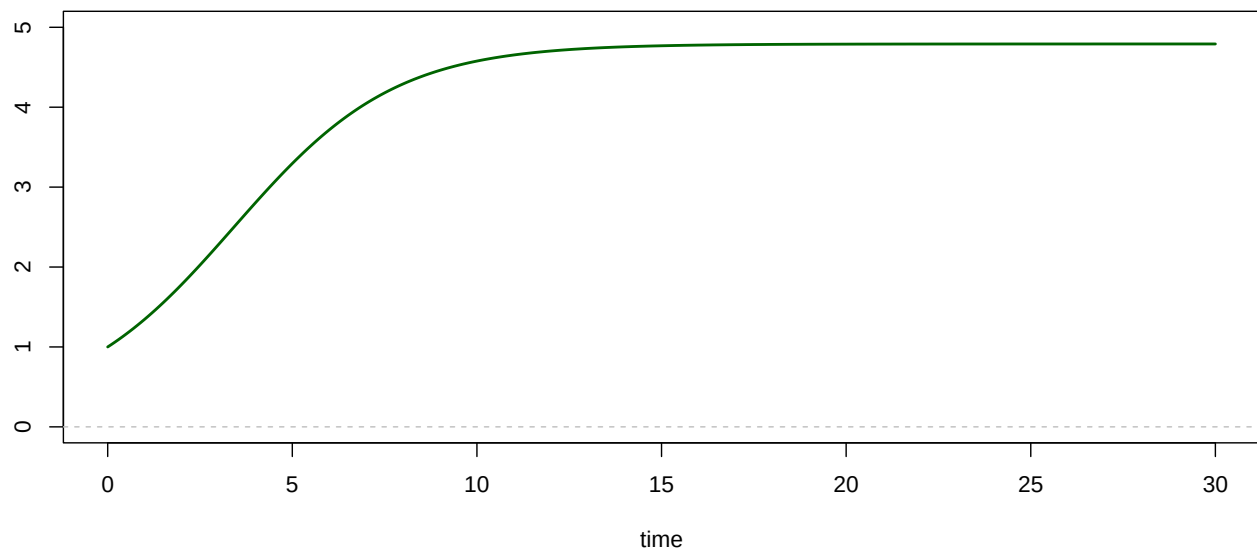
# Graphe Q=0.8
# Note : on ajuste l'échelle y pour voir l'extinction
plot(solQ1, type="l", col="darkred", lwd=2, ylim=c(0, 1.5),
     main="Pêche par quota : Q=0.8\n(Surexploitation)")
abline(h=0, col="gray", lty=2)
```

**Pêche par quota : Q=0.8
(Surexploitation)**



```
# Graphe Q=0.1
plot(solQ2, type="l", col="darkgreen", lwd=2, ylim=c(0, 5),
     main="Pêche par quota : Q=0.1\n(Équilibre)")
abline(h=0, col="gray", lty=2)
```

**Pêche par quota : Q=0.1
(Équilibre)**



```
par(mfrow = c(1, 1))
```